

GIORNO 02

Variabili e tipi di dato: dare un nome alle cose e sapere di che pasta sono fatte

Indice della giornata

Mattina — che cosa sono i dati

1. Ripasso: i quattro debiti di ieri
2. La variabile è un'etichetta, non una scatola
3. Nomi: le regole e il buon gusto
4. Riassegnare, scambiare, e le prime f-string
5. Il tipo di un valore
6. I numeri: interi e con la virgola
7. Vero e falso

Pomeriggio — come si maneggiano e come si stampano

8. Il testo: indici, slicing, metodi
9. Operatori, precedenza e casting
10. Le f-string
11. Costanti e scelta del tipo
12. Errori tipici della giornata

1. RIPASSO E DOMANDE

I quattro debiti di ieri, e la scadenza è adesso

I quattro debiti di ieri

#	Il problema di ieri	La soluzione di oggi	Blocco
1	89.00 viene stampato 89.0: gli zeri finali spariscono	f-string con <code>:.2f</code>	10
2	Per incolonnare si contano gli spazi a mano, e alla quarta riga si sballa	f-string con <code>:>10 :<10 :^10</code>	10
3	"5" + "3" fa 53 e non 8	i tipi e il casting	5 e 9
4	"Totale: " + 42 fa esplodere il programma	f-string, oppure <code>str(42)</code>	9 e 10

I primi due sono problemi di forma: come appare un numero. Gli altri due sono di sostanza: che cosa quel numero è. Cominciamo dalla sostanza.

Tre domande, da rispondere in chat

1. Che cosa stampa questa?

```
print("Colli:", 3, sep="")
```

2. Quante righe stampano queste due?

```
print("A", end="")
```

```
print("B")
```

3. Perché questa si rompe?

```
totale = 244.0
```

```
print("Totale: " + totale)
```

Senza lanciare niente. La terza è la porta d'ingresso della giornata.

Le risposte

Colli:3 — sep="" toglie lo spazio che print mette di suo

Una riga sola, AB — il primo print non va a capo

La terza si rompe con un TypeError

il + non sa mescolare testo e numeri

Ieri la soluzione era la virgola.

Oggi impariamo quella vera, e capiamo perché il problema esiste.

Com'è fatta la giornata

sei blocchi di teoria e cinque laboratori, come ieri

mattina: che cosa sono i dati. Pomeriggio: come si maneggiano

è la giornata più densa delle quattro di fundamenta, non la più difficile

se stasera vi sembra troppa roba, è normale ed è previsto

2. LA VARIABILE È UN'ETICHETTA, NON UNA SCATOLA

Nome e valore sono due cose separate

L'etichetta sul barattolo

"Una variabile è un contenitore che contiene un valore":
comodo per dieci minuti, poi comincia a fare danni.
Il valore non sta dentro il nome: il nome è solo un modo per arrivarci.

Analogia — l'etichetta sul barattolo

la variabile non è il barattolo: è l'etichetta attaccata sopra
staccatela e attaccatela a un altro barattolo:
il primo non è sparito, è ancora in dispensa
solo che nessuno lo chiama più
e se nessuno lo cerca più, prima o poi finisce in cantina

Il "finisce in cantina" è letteralmente quello che fa Python:
libera la memoria dei valori che nessun nome raggiunge più.

Che cosa succede davvero

```
prodotto = "Cuffie Bluetooth"
```

```
prezzo = 49.90
```

```
print(prodotto)          # Cuffie Bluetooth
```

```
print(prezzo)            # 49.9    <-- e voi avevate scritto 49.90
```

Per un numero lo zero finale non esiste. Lo vogliamo noi quando stampiamo.

Riassegnare non modifica niente

```
zona = "NORD"

zona = "SUD"

prezzo_listino = 89.00
prezzo_esposto = prezzo_listino
prezzo_esposto = 79.00

print(zona)          # SUD
print(prezzo_listino) # 89.0  <-- non si è mosso
print(prezzo_esposto) # 79.0
```

La seconda riga non ha modificato "NORD": ha staccato l'etichetta e l'ha spostata. Con i tipi di oggi funziona sempre così; con le liste, quando arriveremo alle strutture dati, diventerà più delicato e ve lo ridirò.

3. NOMI: LE REGOLE E IL BUON GUSTO

Quello che vieta Python e quello che vieta il buon senso

Quello che vieta Python

Regola	Va bene	Non va bene
Comincia con una lettera o _	zona, _temporaneo	2zona, 1cliente
Solo lettere, cifre e _	peso_kg, zona2	peso-kg, peso kg
Non è una parola riservata	da_stampare	class, for, if, import
Maiuscole e minuscole contano	zona e Zona sono due nomi diversi	—

1cliente = "Rossi" dà SyntaxError: invalid decimal literal. Python legge da sinistra: appena vede una cifra smette di aspettarsi un nome.

Le parole riservate che rischiate davvero di usare

Parola	A che cosa serve	Quando la vedremo
if, else, elif	Decisioni	con le condizioni
for, while, break, continue	Cicli	con i cicli
def, return	Funzioni	con le funzioni
in, and, or, not	Appartenenza e logica	con le condizioni
True, False, None	Valori speciali	oggi i primi due
class, import, try, lambda	Cose che qui non useremo	mai in questo blocco

L'editor le colora diversamente: se un vostro nome si colora come una parola chiave, cambiatelo. print, type, len, str e int invece non sono riservate: Python vi lascia usarle, ed è una pessima idea che costa un pomeriggio.

Quello che vieta il buon senso

Cattivo	Buono	Perché
x	peso_kg	Non dice niente, e nemmeno l'unità di misura
dato1, dato2	nome_cliente, codice_fiscale	I numeri progressivi non spiegano nulla
nu	numero_utenti	Le abbreviazioni le capite solo voi, e solo oggi
tmp	totale_parziale	"Temporaneo" resta lì per anni
prezzoConIva	prezzo_con_iva	Stile di un altro linguaggio
lista_prezzi per un prezzo solo	prezzo	Il nome mente sul contenuto

L'ultima riga è la più importante: un nome che mente è peggio di un nome che non dice niente. x vi costringe ad andare a guardare, lista_prezzi vi illude.

I nomi in questo corso

Criterio unico: leggete la riga ad alta voce.

Se si capisce senza risalire al punto in cui la variabile è nata, il nome è buono.

Le regole del corso

tutto minuscolo con underscore: `prezzo_finale`

nomi in italiano

costanti tutte maiuscole in cima al file

niente accenti: si scrive `eta`, non `età`

Perché niente accenti

Python li accetta, ma `eta` ed `età` sono due nomi diversi.

4. RIASSEGNARE, SCAMBIARE, E LE PRIME F-STRING

L'= si legge da destra verso sinistra

La riga che sembra un errore di matematica

```
contatore = 10  
contatore = contatore + 1  
  
print(contatore)          # 11
```

L' = non vuol dire "è uguale a": Python calcola 11, poi ci attacca l'etichetta.

Le forme abbreviate

# totale = totale + 5	si scrive	totale += 5	aumenta di 5
# totale = totale - 5	si scrive	totale -= 5	diminuisce di 5
# totale = totale * 2	si scrive	totale *= 2	raddoppia

```
totale_ordine = 100.00
totale_ordine += 27.00
totale_ordine -= 10.00
totale_ordine *= 2
print(totale_ordine)      # 234.0
```

```
codice = "LS"
codice += "-2026"
codice += "-MI"
print(codice)             # LS-2026-MI
```

+= non è =+. La riga totale += 5 non dà errore: assegna 5 e butta via il resto.

Assegnare a più variabili, e scambiare

```
nome, cognome = "Giulia", "Ferrante"
```

```
print(nome)           # Giulia
```

```
print(cognome)        # Ferrante
```

```
zona_partenza = "NORD"
```

```
zona_arrivo = "SUD"
```

```
zona_partenza, zona_arrivo = zona_arrivo, zona_partenza
```

```
print(zona_partenza)  # SUD
```

```
print(zona_arrivo)    # NORD
```

Lo scambio funziona perché Python calcola prima tutta la parte destra: in altri linguaggi servono tre righe e una variabile d'appoggio. Usatela quando i valori sono davvero legati, come nome e cognome.

Le f-string, versione minima

```
nome = "Giulia"
```

```
prezzo = 89.00
```

```
print(f"Cliente: {nome}")          # Cliente: Giulia
```

```
print(f"Prezzo: {prezzo}")         # Prezzo: 89.0
```

```
print(f"Prezzo: {prezzo:.2f}")     # Prezzo: 89.00    <-- primo debito saldato
```

```
# La f va PRIMA delle virgolette: senza, Python stampa le graffe e tace.
```

```
# Dentro le graffe il nome va SENZA virgolette: {nome} dà Giulia.
```

Per oggi vi bastano {variabile} e {variabile:.2f}.

5. IL TIPO DI UN VALORE

Di che pasta è fatto il dato

type(): lo strumento

```
print(type("Cuffie Bluetooth"))    # <class 'str'>
print(type(49))                    # <class 'int'>
print(type(49.90))                 # <class 'float'>
print(type(True))                   # <class 'bool'>
```

La parola class non c'entra con gli oggetti, che qui non facciamo: leggetela "genere".

I quattro tipi di oggi

Tipo	Nome per esteso	Che cosa contiene	Esempi
int	numero intero	numeri senza virgola, anche negativi	0, 42, -7, 2026
float	numero con la virgola	numeri con parte decimale	49.9, 0.22, -3.5, 5.0
bool	booleano	due soli valori possibili	True, False
str	stringa (testo)	qualunque sequenza di caratteri	"Mario", "42", ""

Quattro tipi, e per oggi bastano: coprono quasi tutto quello che passa per un ufficio. Il nome del cliente è str, i colli spediti int, il peso della spedizione float, "la fattura è già pagata" bool.

Il tipo appartiene al valore, non al nome

`dato = 42` e due righe dopo `dato = "quarantadue"`

il tipo passa da `int` a `str` e nessuno protesta

La variabile non ha un tipo: ce l'hanno i valori a cui è attaccata di volta in volta. Si chiama tipizzazione dinamica.

comoda: scrivete meno e provate più in fretta

pericolosa: nessuno vi ferma se mettete un testo dove c'era un numero,
e il programma si rompe più avanti, dove l'errore non c'entra niente

Che cosa fate voi, in pratica

date nomi che dicano il tipo: `prezzo_testo` e `prezzo_numero`

non cambiate tipo a una variabile a metà programma

non è vietato: è maleducato

Che tipo esce da un calcolo?

Provate a indovinare prima di guardare la colonna di destra.

```
print(type(3 + 4))      # <class 'int'>
print(type(3 + 4.0))    # <class 'float'>    int promosso a float
print(type(10 / 2))     # <class 'float'>    10 / 2 fa 5.0, non 5
print(type(10 // 2))    # <class 'int'>
print(type("3" + "4"))  # <class 'str'>      non è una somma, è un incollaggio
print(type(3 > 4))      # <class 'bool'>     il confronto risponde sì o no
```

La terza è quella che frega di più: la divisione / dà sempre un float. Il tipo è di che materiale è fatto il dato, non a cosa serve: due prezzi si sommano, due codici fiscali no, anche se sono pieni di cifre.

6. I NUMERI: INTERI E CON LA VIRGOLA

int, float, e la regola sui soldi

int: grandezza illimitata, e l'underscore

```
abitanti = 1_250_000
```

```
print(2 ** 10)          # 1024
```

```
print(2 ** 100)         # 1267650600228229401496703205376
```

```
print(abitanti)         # 1250000
```

Un int non ha tetto: contate centesimi, righe di un file, occorrenze senza chiedervi se il contatore esploderà. Underscore e non punti, da cinque cifre in su: 1.250.000 non è un numero, è un errore.

// % ** al lavoro

```
# //  divisione intera      17 // 5  ->  3
# %   resto (modulo)       17 % 5   ->  2
# **  elevamento a potenza 2 ** 8   -> 256
```

```
minuti_totali = 197
```

```
lato_cm = 45
```

```
print(minuti_totali // 60)  # 3      quante volte il 60 ci sta dentro
print(minuti_totali % 60)   # 17     quello che avanza
print(11 % 2)               # 1      0 per i pari, 1 per i dispari
print(lato_cm ** 3)         # 91125  il volume
print(lato_cm * 3)          # 135    il lato per tre
```

Perché $0.1 + 0.2$ non fa 0.3

```
print(0.1 + 0.2)          # 0.30000000000000004
print(0.1 + 0.2 == 0.3)   # False
print(1.1 + 2.2)          # 3.3000000000000003
print(2.00 - 1.10)        # 0.8999999999999999
```

L'ultima è il resto di chi paga con due euro un articolo da 1.10. Ed esce sbagliato.

Il righello con le tacche

Non è un bug di Python: fanno la stessa cosa Java, C, JavaScript, Excel (che però ve lo nasconde) e la calcolatrice del telefono.

Analogia — il righello con le tacche

un float è un righello con un numero finito di tacche
fra una tacca e l'altra non può misurare: sceglie la più vicina
0.1 in binario cade fra due tacche, e Python usa quella accanto
sommate tre di quei capelli e il risultato si vede

Il punto pratico non è capire il binario.

È non fidarsi mai di un `==` fra due numeri con la virgola.

I decimali si scrivono con il punto: 12.5, mai 12,5.

12,5 non dà errore: sono due valori, e i conti escono assurdi in silenzio.

round(), e due sorprese

```
print(round(3.14159, 2)) # 3.14
print(round(3.7))       # 4
print(round(0.5))       # 0
print(round(1.5))       # 2
print(round(2.5))       # 2      <-- non 3
print(round(3.5))       # 4
print(round(2.675, 2))  # 2.67   <-- non 2.68
```

Con due argomenti arrotonda a quel numero di decimali.

Con uno solo arrotonda all'intero e restituisce un int.

A metà esatta Python va al pari più vicino: è l'arrotondamento bancario, serve a non gonfiare i totali quando si arrotondano migliaia di valori, e non si può disattivare. 2.675 è conservato appena sotto, e round() arrotonda quello che c'è davvero.

I soldi si contano in centesimi interi

```
CENTESIMI_IN_UN_EURO = 100
```

```
totale_centesimi = 110 + 220
```

```
print(totale_centesimi) # 330
```

```
print(totale_centesimi / CENTESIMI_IN_UN_EURO) # 3.3
```

```
# È il trucco del cassiere: contare le monetine da un centesimo
```

```
# invece dei decimali. Con gli interi "uguale" vuol dire davvero uguale.
```

Si divide per cento solo al momento di stampare: sono gli esercizi 02.3 e 02.6. Per i soldi esiste anche un tipo apposito, Decimal, ma richiede un import e questo blocco non importa niente: così ogni file gira su qualunque computer.

7. VERO E FALSO

Un confronto non decide niente: produce un valore

I sei confronti

Operatore	Si legge	Esempio	Risultato
==	è uguale a	18 == 18	True
!=	è diverso da	18 != 21	True
<	è minore di	10 < 5	False
>	è maggiore di	10 > 5	True
<=	è minore o uguale a	10 <= 10	True
>=	è maggiore o uguale a	9 >= 10	False

= assegna, e si legge "prende il valore". == confronta, e si legge "è uguale a". Oggi scambiarsi non fa danni visibili perché non c'è ancora if. Con le condizioni sì.

Un confronto produce un valore

`eta >= 18` non è una domanda e non è una condizione.

È un'espressione che produce un valore, come `2 + 3` produce 5.

Solo che il valore prodotto è `True` oppure `False`.

Siccome è un valore:

- si può mettere in una variabile

- si può stampare

- si può mettere dentro le graffe di una f-string

Quello che oggi non facciamo è farci prendere una decisione dal programma.

Per quello serve `if`, che arriva quando affronteremo le condizioni.

Chi arriva a `if` senza aver mai visto un booleano da solo
impara due cose insieme e ne capisce una.

Booleani da guardare

```
eta = 34

SOGLIA_MAGGIORE_ETA = 18

spesa = 1284.50

SOGLIA_TOP = 1000.00

maggiorenne = eta >= SOGLIA_MAGGIORE_ETA

print(maggiorenne)                # True

print(type(maggiorenne))          # <class 'bool'>

print(f"Cliente top: {spesa > SOGLIA_TOP}")  # Cliente top: True
```

Nessun if in questa slide: oggi i booleani si producono e si guardano.

Le righe da leggere due volte

```
print("SUD" == "sud")      # False  <-- maiuscole e minuscole contano
print("SUD" == " SUD")    # False  <-- uno spazio invisibile in testa
print("abaco" < "abete")   # True
print("Zebra" < "abaco")   # True   <-- le maiuscole vengono prima delle minuscole
print(89 == 89.0)         # True
print(10 == "10")         # False  <-- terzo debito: il numero non è il testo
print(True == 1)         # True
print(bool(0))            # False
print(bool(""))           # False
print(bool("0"))          # True   <-- un carattere, quindi non è vuota
print(bool(" "))          # True   <-- anche lo spazio è un carattere
```

Prima di confrontare due testi si normalizza: `.lower()` per le maiuscole, `.strip()` per gli spazi. `bool()`: è falso lo zero e il vuoto, è vero tutto il resto, e le ultime due righe sono trappole.

8. IL TESTO

len, indici, slicing, e i metodi che userete davvero

Apici, testo su più righe, len()

```
NOTA = """Presentarsi 15 minuti prima.  
Portare tessera sanitaria e impegnativa."""  
  
print(NOTA)  
  
print(len("Battipaglia"))      # 11  
  
print(len("  mario  "))      # 9    <-- len conta anche gli spazi
```

Se dentro c'è un apostrofo, fuori le doppie; i tripli apici tengono gli a capo. len() conta anche gli spazi: è così che si scoprono quelli invisibili in coda a un dato importato male, ed è quello che farete in 02.4.

Le posizioni dei caratteri

```
stringa:  L  S  -  2  0  2  6  -  M  I  -  0  0  4  2
indice:    0  1  2  3  4  5  6  7  8  9 10 11 12 13 14
da destra:                -4 -3 -2 -1
```

```
Traceback (most recent call last):
  File "/Users/giulia/corso-python/giorno_02/codice.py", line 3, in <module>
    print(codice[20])
    ~~~~~^
IndexError: string index out of range
```

Si comincia a contare da zero: una stringa lunga 15 ha indici da 0 a 14. Gli indici negativi contano da destra: -1 è l'ultimo, -2 il penultimo.

Un carattere e un pezzo

```
codice = "LS-2026-MI-0042"
```

```
print(codice[0])      # L
print(codice[8])      # M
print(codice[-1])     # 2
print(codice[0:2])     # LS
print(codice[3:7])     # 2026
print(codice[:2])      # LS      dall'inizio
print(codice[11:])     # 0042    fino alla fine
print(codice[-4:])     # 0042    gli ultimi quattro
```

Leggete codice[0:2] così: da 0, per 2 caratteri. La differenza è la lunghezza del pezzo.

Le tre cose da sapere sullo slicing

1. L'estremo destro è escluso

`codice[0:2]` prende i caratteri 0 e 1, non 0, 1 e 2

è la cosa che si sbaglia sempre, anche dopo anni

2. I due estremi si possono omettere

`[:2]` vuol dire dall'inizio, `[11:]` vuol dire fino alla fine

`[-4:]` vuol dire gli ultimi quattro, senza sapere quanto è lunga

ed è la forma più robusta

3. Lo slicing non si rompe se sforate

`"LS"[0:100]` restituisce LS e tace

comodo e insidioso: un errore di conteggio non produce un errore,

produce un pezzo sbagliato

La stringa non si può modificare

```
nome = "mario"

print(nome.upper())      # MARIO

print(nome)              # mario    <-- non è cambiato niente

nome_pulito = nome.upper()

print(nome_pulito)       # MARIO
```

È scolpita nella pietra: .upper() ne intaglia una copia nuova e l'originale resta lì. Negli esercizi teniamo separati il dato grezzo e quello pulito: serve a stampare il prima e il dopo quando si bonifica un archivio.

Ripetere una stringa

```
print("=" * 40)
```

```
print("#" * 12)
```

```
print("ab" * 3)
```

```
# =====
```

```
# #####
```

```
# ababab
```

```
# Il moltiplicatore deve essere un intero:
```

```
# "#" * 12.4 -> TypeError: can't multiply sequence by non-int of type 'float'
```

Da oggi nessuna riga di separazione si scrive per esteso contando i caratteri. E in 02.7, per la lunghezza di una barra, si usa // e non /: il moltiplicatore deve restare un intero.

Che cos'è un metodo

Un metodo è una funzione attaccata a un valore.

prima il valore, poi il punto, poi il nome, poi le parentesi

le parentesi ci vanno sempre, anche se vuote

nome.strip senza parentesi non esegue niente e non dà nemmeno errore

I metodi funzionano solo sul tipo giusto

numero.upper() dà `AttributeError: 'int' object has no attribute 'upper'`

tradotto: gli interi non sanno fare upper

ed è corretto, perché le maiuscole di 42 non esistono

len() invece è una funzione, non un metodo

si scrive len(nome), non nome.len()

Pulire e uniformare

Metodo	Che cosa fa	"mario de luca" diventa
<code>.strip()</code> <code>.lstrip()</code> <code>.rstrip()</code>	toglie gli spazi ai bordi, da entrambi i lati o da uno solo	—
<code>.upper()</code>	tutto maiuscolo	MARIO DE LUCA
<code>.lower()</code>	tutto minuscolo	mario de luca
<code>.title()</code>	prima lettera di ogni parola maiuscola	Mario De Luca
<code>.capitalize()</code>	prima lettera della stringa maiuscola, resto minuscolo	Mario de luca
<code>.replace(vecchio, nuovo)</code>	sostituisce tutte le occorrenze, non la prima	—

Per un nome proprio `.title()`, per un ruolo o una pratica `.capitalize()`: "Infermiera Di Ricerca" sembra il titolo di un giornale americano. E `.title()` sui cognomi italiani va sempre riguardato: "dell'orto" diventa "Dell'Orto".

Ripulire davvero

```
grezzo = "    Poliambulatorio Aurora    "

nome = "mario de luca"

prezzo_italiano = "1.234,56"


print(f"[{grezzo}]")           # [    Poliambulatorio Aurora    ]
print(f"[{grezzo.strip()}]")   # [Poliambulatorio Aurora]
print(f"[{grezzo.rstrip()}]")  # [    Poliambulatorio Aurora]
print(nome.title())           # Mario De Luca
print(nome.capitalize())      # Mario de luca
print(prezzo_italiano.replace(".", "").replace(",", " "))          # 1234.56
print("089 / 12 34 56".replace(" ", ""))                          # 089/123456
```

Le quadre attorno al valore sono un trucco per la vita: gli spazi non si vedono, e senza un delimitatore non sapete se ci sono. Nei due replace in fila l'ordine conta: prima i punti delle migliaia, poi la virgola decimale.

split: spaccettare un codice

```
codice = "LS-2026-MI-0042"

prefisso, anno, provincia, progressivo = codice.split("-")

print(prefisso)      # LS
print(anno)          # 2026
print(provincia)     # MI
print(progressivo)   # 0042
```

Quattro nomi a sinistra e quattro pezzi a destra: se non corrispondono, il programma si ferma. Quello che split restituisce è un elenco di pezzi, e sa fare molte più cose di così: le vedrete quando arriveremo alle strutture dati.

Cercare e verificare

```
email = "mario.deluca@comune.villanova.it"
```

```
print(email.startswith("mario"))    # True
print(email.endswith(".it"))        # True
print(email.count("@"))              # 1      una sola chiocciola
print(email.find("@"))               # 12     dove sta
print(email.find("SS"))              # -1     non trovato, e non è un errore
print("0042".isdecimal())            # True
print("3.5".isdecimal())             # False  <-- il punto non è una cifra
print("").isdecimal())               # False  <-- il vuoto non è un numero
print("Mario1".isalpha())            # False
print("LS2026".isalnum())            # True
```

isdecimal() non verifica che sia un numero: verifica che sia fatto di sole cifre, quindi anche "-7" è False. Da domani, sul dato digitato dall'utente, sono la prima difesa contro chi scrive venti invece di 20.

Concatenare i metodi

```
grezzo = "  MARIO.DELUCA@Comune.Villanova.IT  "
```

```
cognome_grezzo = "de  luca  "
```

```
print(f"[{grezzo.strip().lower()}]")
```

```
# [mario.deluca@comune.villanova.it]
```

```
print(f"[{cognome_grezzo.strip().replace('  ', ' ').title()}]")
```

```
# [De Luca]
```

```
# Con .title() prima di .replace() il doppio spazio resta dentro: De  Luca
```

L'ordine conta: prima si sistema il contenuto, poi la forma. strip() per primo, sempre.

9. OPERATORI, PRECEDENZA E CASTING

Chi va per primo, e come si cambia il tipo di un dato

La precedenza

Priorità	Operatori	Esempio
1	() parentesi	$(2 + 3) * 4 \rightarrow 20$
2	** potenza	$2 * 3 ** 2 \rightarrow 18$
3	* / // %	$2 + 3 * 4 \rightarrow 14$
4	+ -	$10 - 2 - 3 \rightarrow 5$
5	== != < > <= >=	$2 + 3 > 4 \rightarrow \text{True}$

*I confronti hanno priorità più bassa dell'aritmetica: $2 + 3 > 4$ calcola prima $2 + 3$ e poi confronta $5 > 4$. E + e * cambiano mestiere secondo il tipo degli operandi.*

Precedenza al lavoro, e le parentesi in più

```
print(2 + 3 * 4)      # 14
print((2 + 3) * 4)    # 20
print(2 * 3 ** 2)     # 18    <-- prima 3 ** 2, poi 2 * 9. Non 36
print(100 - 20 / 2)   # 90.0
print(2 + 3 > 4)      # True
```

```
CAPITALE = 530000
```

```
TASSO = 6
```

```
MESI = 28
```

```
interessi = (CAPITALE * TASSO * MESI) // (100 * 12)
```

```
print(interessi)      # 74200
```

*Le parentesi al numeratore non servirebbero: costano due caratteri e tolgono ogni dubbio a chi rilegge, voi compresi fra un mese. Senza, bisogna ricordarsi che // ha la stessa priorità di * e che si procede da sinistra.*

Il casting: il gesto esplicito

```
numero = 42

testo = str(42)

print(int("42"))      # 42
print(float("3.5"))   # 3.5
print(bool(0))         # False
print(type(numero))   # <class 'int'>
print(type(testo))    # <class 'str'>
print(numero + 1)     # 43      somma
print(testo + "1")    # 421     incolla
```

È il cambio valuta in aeroporto: bisogna chiederlo, e a volte lo sportello rifiuta.

Le conversioni, una per una

Conversione	Risultato	Nota
<code>int("42")</code>	42	funziona solo con sole cifre
<code>int(" 42 ")</code>	42	tollera gli spazi ai bordi
<code>int("3.5")</code>	ValueError	il punto non è una cifra: <code>int()</code> non arrotonda mai una stringa
<code>int("")</code>	ValueError	il vuoto non è un numero
<code>float("3.5")</code>	3.5	il punto sì, la virgola no
<code>float("3,5")</code>	ValueError	serve prima <code>.replace(",", ".")</code>
<code>int(3.9)</code>	3	taglia, non arrotonda; <code>int(-3.9)</code> fa -3
<code>float(42)</code>	42.0	sempre possibile
<code>str(42.5)</code>	"42.5"	sempre possibile
<code>bool("False")</code>	True	non è vuota, quindi è vera: <code>bool()</code> non legge il contenuto

Le righe con ValueError arrivano tutte dallo stesso posto: un dato scritto a mano da una persona, su un modulo o in un vecchio gestionale, che nessuno ha ripulito prima di passarvelo. L'ultima riga è cattiva e va guardata due volte.

Da "3.5" a intero: due tappe, due risultati

```
testo = "3.5"
```

```
print(int(3.9))          # 3    sui numeri int() taglia
```

```
print(round(3.9))        # 4    round() arrotonda
```

```
print(int(float(testo))) # 3
```

```
print(round(float(testo))) # 4
```

```
# ValueError: invalid literal for int() with base 10: '3.5'
```

```
# Tradotto: "mi hai chiesto di leggere 3.5 come intero in base dieci, non ci riesco".
```

Nel dubbio: sui conteggi si taglia, sui pesi e sulle misure si arrotonda.

La virgola italiana, e il codice che resta testo

```
prezzo_dal_modulo = "120,00"
prezzo = float(prezzo_dal_modulo.replace(",", ".", "."))

codice_articolo = "0042"
numero = int(codice_articolo)

print(prezzo)           # 120.0
print(f"{prezzo:.2f}")  # 120.00
print(numero)           # 42      <-- gli zeri davanti non tornano più
```

Un codice si tiene come testo: 42 non è 0042, e in magazzino non si trova più.

10. LE F-STRING

Come appare un valore, senza toccare il valore

:.2f — i decimali fissi

```
prezzo = 89.00
```

```
iva = 16.643
```

```
zero = 0.0
```

```
print(f"{prezzo}")           # 89.0
```

```
print(f"{prezzo:.2f}")       # 89.00
```

```
print(f"{iva:.2f}")          # 16.64
```

```
print(f"{iva:.0f}")          # 17
```

```
print(f"{zero:.2f}")         # 0.00
```

.2f = numero con la virgola, esattamente due decimali.

Non uno di meno se sono zeri, non uno di più se ce ne sono altri.

Primo debito saldato: ogni importo in euro si stampa con :.2f. Arrotonda per mostrare.

Le colonne, senza contare gli spazi

```
print(f"[{'Cuffie':<12}]\n")
print(f"[{'Cuffie':>12}]\n")
print(f"[{'Cuffie':^12}]\n")
print(f"{'Cuffie':<20}{49.90:>10.2f}\n")
print(f"{'Monitor 27 pollici':<20}{219.00:>10.2f}\n")
print(f"{'Hub USB-C':<20}{27.00:>10.2f}\n")
```

```
[Cuffie      ]
[      Cuffie]
[  Cuffie   ]
Cuffie                49.90
Monitor 27 pollici    219.00
Hub USB-C             27.00
```

Secondo debito saldato. < a sinistra per i testi, > a destra per i numeri, ^ al centro per i titoli. La larghezza è un minimo, non un massimo: quando una riga vi sballa la tabella, la causa è quasi sempre un nome più lungo del previsto.

Migliaia, percentuali, larghezza variabile

```
LARGHEZZA = 12

vendite = 1234567

incidenza = 0.262456

prezzo = 89.00

quantita = 3


print(f"{vendite:,}")          # 1,234,567
print(f"{vendite:>15,}")       #          1,234,567
print(f"{incidenza:.1%}")     # 26.2%
print(f"[{'Cuffie':<{LARGHEZZA}}]") # [Cuffie      ]
print(f"[{49.90:>{LARGHEZZA}.2f}]") # [          49.90]
print(f"Totale: {prezzo * quantita:.2f} EUR") # Totale: 267.00 EUR
print(f"Sopra i 200: {prezzo * quantita > 200}") # Sopra i 200: True
print(f"Barra: {'#' * quantita}") # Barra: ###
```

Il separatore delle migliaia è la virgola, all'americana: con questa sintassi non si cambia, e lo teniamo così. E % moltiplica già per cento: non fatelo anche voi, o uscite con 1500.0%.

Il promemoria delle f-string

Scrivete	Con valore = 1234.5 ottenete	Serve per
{valore}	1234.5	vedere il valore grezzo
{valore:.2f}	1234.50	importi in euro
{valore:>12}	1234.5	colonna a destra
{valore:<12}	1234.5	colonna a sinistra
{valore:^12}	1234.5	titoli e badge
{valore:,}	1,234.5	numeri grandi
{valore:>12,.2f}	1,234.50	la forma completa per gli importi
{0.256:.1%}	25.6%	percentuali
{'testo':<10}	testo	etichette a larghezza fissa

L'ordine dentro le graffe è sempre: allineamento, larghezza, virgola, decimali. Dentro le graffe di una f-string a virgolette doppie si usano gli apici singoli.

11. COSTANTI E SCELTA DEL TIPO

Niente numeri magici, e le tre domande sul tipo

Che cosa vuol dire 0.22?

```
totale = 200.00 + 200.00 * 0.22
```

```
sconto = 200.00 * 0.15
```

```
print(totale)    # 244.0
```

```
print(sconto)   # 30.0
```

```
# Quel 0.22 è l'IVA o un margine?
```

```
# Quel 0.15 è uno sconto o una provvigione?
```

```
# Quando l'IVA cambierà, quanti punti del file dovreste toccare?
```

```
# E siete sicuri di trovarli tutti?
```

Un numero in mezzo al codice senza un nome si chiama magic number: da noi è vietato.

La soluzione: dare un nome

```
ALIQUOTA_IVA = 0.22

PERCENTUALE_SCONTO = 0.15


imponibile = 200.00

iva = imponibile * ALIQUOTA_IVA

sconto = imponibile * PERCENTUALE_SCONTO


print(f"IVA:      {iva:.2f}")      # IVA:      44.00
print(f"Sconto:   {sconto:.2f}")   # Sconto:   30.00
```

Si legge, si cambia in un posto solo, e se sbagliate a digitare il programma si ferma.

Costanti calcolate da altre costanti

```
LARGHEZZA_BADGE = 44
LARGHEZZA_INTERNA = LARGHEZZA_BADGE - 2
BORDO = "#" * LARGHEZZA_BADGE

print(BORDO)
print("#" + f"{'CONGRESSO':^{LARGHEZZA_INTERNA}}" + "#")
print(BORDO)

#####
#                CONGRESSO                #
#####
```

Cambiate 44 in 60 e il riquadro si adatta da solo. È il test dell'esercizio 02.8: se cambiando una costante il riquadro si rompe, da qualche parte avete contato a mano.

Che cosa va e che cosa non va in costante

Va in costante	Non va in costante
Aliquote, tariffe, soglie	I dati del singolo cliente
Larghezze di colonna, caratteri di bordo	I risultati dei calcoli
Formati fissi ("LS-", lunghezza attesa)	Le variabili che cambiano nel programma
Fattori di conversione	I valori che arrivano dall'esterno

La costante vale per tutte le pratiche, la variabile vale per questa pratica qui. E i risultati dei calcoli non si scrivono mai a mano: si scrive il calcolo.

Le tre domande per scegliere il tipo

Davanti a un dato nuovo, in quest'ordine:

1. Ci devo fare dei conti?

se no, è testo. Punto.

2. Può avere una parte decimale?

se sì è float, se no è int

3. Sono soldi?

allora sono centesimi, e quindi int

La prima domanda risolve quasi tutti i casi ed è quella che si sbaglia di più, perché l'istinto guarda l'aspetto del dato invece dell'uso.

Tabella di decisione (1/2)

Dato	Tipo	Perché
Nome del paziente	str	è testo
Codice fiscale	str	contiene lettere, e non ci si fanno conti
CAP 84091	str	04100 perderebbe lo zero iniziale
Numero di telefono	str	comincia per zero, può avere il +39
Quantità ordinata	int	si conta, non esistono mezzi pezzi
Prezzo unitario	float per mostrarlo, int in centesimi per calcolarlo	blocco 6
Peso in chili	float	esistono i 18,5 kg

Tabella di decisione (2/2)

Dato	Tipo	Perché
Aliquota IVA	float	0.22
Percentuale di sconto	float	0.15, mai 15
Anno	int	si fa la differenza fra due anni
Data 01/03/1980	str (per oggi)	tre numeri e due barre
Ha la carta fedeltà	bool	ha due soli valori
Codice articolo LS-2026-MI-0042	str	contiene lettere e trattini
Progressivo 0042	str per il codice, int per contare	gli zeri iniziali contano

I tre casi che ingannano tutti

Il numero che non è un numero

CAP, telefono, codice fiscale, partita IVA, IBAN, codice articolo
sembrano numeri e sono etichette
il test è brutale: ha senso sommarne due? Se no, è testo

Il decimale che non deve essere decimale

i soldi: sembrano il caso da manuale per float,
e sono il caso da manuale in cui float fa danni

Il booleano scritto come testo

"SI", "NO", "True", "1": arrivano così dai sistemi vecchi
finché restano testo, qualcuno confronta "SI" con "Si" e ottiene False
vanno convertiti appena entrano nel programma, non a metà

12. ERRORI TIPICI DELLA GIORNATA

Leggere il traceback, e il metodo che funziona sempre

TypeError: testo più numero

Traceback (most recent call last):

File "/Users/giulia/corso-python/giorno_02/cassa.py", line 3, in <module>

print("Totale: " + totale)

~~~~~^~~~~~

TypeError: can only concatenate str (not "float") to str

*L'errore più frequente di oggi, ed è lo stesso di ieri. Ultima riga: che errore è. Riga File: dove.*

# Le tre soluzioni, in ordine di preferenza

```
totale = 244.0
```

|                                            |                             |                      |
|--------------------------------------------|-----------------------------|----------------------|
| <pre>print(f"Totale: {totale:.2f}")</pre>  | <pre># Totale: 244.00</pre> | quella di oggi       |
| <pre>print("Totale: " + str(totale))</pre> | <pre># Totale: 244.0</pre>  | funziona, ma verbosa |
| <pre>print("Totale:", totale)</pre>        | <pre># Totale: 244.0</pre>  | quella di ieri       |

*Quarto debito saldato. La prima risolve anche il problema della formattazione.*

# ValueError: la conversione rifiutata

```
Traceback (most recent call last):  
  File "/Users/giulia/corso-python/giorno_02/importa.py", line 2, in <module>  
    quantita = int(quantita_grezza)  
                ^^^^^^^^^^^^^^^^^^^  
ValueError: invalid literal for int() with base 10: '3,5'
```

*TypeError è "mi hai dato la cosa sbagliata"; ValueError è "la cosa è giusta ma il contenuto no". Le tre cause: la virgola al posto del punto, gli spazi, il testo vero.*

# I due errori che non danno errore

```
nome_sbagliato = "  mario  "
nome_sbagliato.strip()
print(f"[{nome_sbagliato}]")      # [  mario  ]    la copia pulita è stata buttata
```

```
nome_giusto = "  mario  "
nome_giusto = nome_giusto.strip()
print(f"[{nome_giusto}]")        # [mario]
```

```
prezzo = 89.00
print("Prezzo: {prezzo:.2f}")     # Prezzo: {prezzo:.2f}    manca la f davanti
```

*Nessun traceback e il programma fa la cosa sbagliata: sono i due più insidiosi.*

# Pronto soccorso

| Messaggio                                     | Che cosa è successo            | Prima cosa da guardare                |
|-----------------------------------------------|--------------------------------|---------------------------------------|
| TypeError: can only concatenate str           | + fra testo e numero           | usate una f-string                    |
| TypeError: can't multiply sequence by non-int | "#" * 12.4                     | serve // invece di /                  |
| TypeError: unsupported operand type(s) for -  | manca un int()                 | il dato è testo e lo credevate numero |
| ValueError: invalid literal for int()         | testo non numerico             | virgola, spazi, testo vero            |
| ValueError: could not convert string to float | idem per float()               | serve .replace(",", ".")              |
| AttributeError: 'int' object has no attribute | metodo di stringa su un numero | serve str() prima                     |
| IndexError: string index out of range         | indice oltre la fine           | si conta da zero; c'è [-1]            |
| NameError: name 'x' is not defined            | nome mai creato                | battitura, maiuscole, ordine          |
| Nessun errore, valore sbagliato               | assegnazione dimenticata       | nome = nome.strip()                   |
| Grafte stampate a schermo                     | manca la f                     | print(f"...")                         |

# Il metodo, sempre lo stesso

## 1. Leggete

ultima riga: che errore è. Riga File: dove.

## 2. Chiedetevi di che tipo è il dato su quella riga

nove errori su dieci di oggi sono un tipo diverso da quello che credevate

## 3. Stampate il tipo

`print(type(sospetto))` è la mossa da fare per prima quando qualcosa non torna  
costa una riga e risolve la giornata

## 4. Una modifica alla volta, poi rilanciate

Il punto 3 è la novità di oggi: è l'unico strumento diagnostico  
che avete finché non arrivano gli altri.

# RIEPILOGO ED ESERCIZI

---

Che cosa portate a casa, e su che cosa si lavora

# Riepilogo del giorno (1/3)

| Argomento      | Concetti chiave                                                                                                                         |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Variabile      | È un'etichetta attaccata a un valore, non un contenitore; riassegnare sposta l'etichetta                                                |
| Nomi           | Lettera o _ iniziale, niente parole riservate, snake_case, in italiano, niente accenti                                                  |
| Riassegnazione | $x = x + 1$ si legge da destra; $+=$ $-=$ $*=$ ; $a, b = 1, 2$ ; scambio $a, b = b, a$                                                  |
| Tipo           | <code>type()</code> risponde <code>&lt;class 'int'&gt;</code> ; il tipo appartiene al valore, non al nome                               |
| int            | Grandezza illimitata; <code>//</code> quoziente, <code>%</code> resto, <code>**</code> potenza; <code>-7 // 2</code> fa <code>-4</code> |
| float          | Punto e non virgola; <code>0.1 + 0.2</code> non fa <code>0.3</code> ; <code>round()</code> va al pari più vicino                        |
| Soldi          | Si contano in centesimi interi e si divide per cento solo alla stampa                                                                   |

## Riepilogo del giorno (2/3)

| Argomento   | Concetti chiave                                                                                          |
|-------------|----------------------------------------------------------------------------------------------------------|
| bool        | True / False; i confronti producono un booleano; oggi si stampano, l'uso nelle decisioni arriva con l'if |
| = contro == | = assegna, == confronta; 10 == "10" è False, 89 == 89.0 è True                                           |
| bool()      | È falso lo zero e il vuoto, vero tutto il resto; bool("False") è True                                    |
| str         | len(), indici da 0, indici negativi, slicing con estremo destro escluso; immutabile                      |
| Ripetizione | "=" * 40; il moltiplicatore deve essere un intero                                                        |
| Metodi      | strip upper lower title capitalize replace split startswith find isdecimal; strip() per primo            |

# Riepilogo del giorno (3/3)

| Argomento        | Concetti chiave                                                                            |
|------------------|--------------------------------------------------------------------------------------------|
| Precedenza       | () poi ** poi * // % poi + - poi i confronti; parentesi anche se non servono               |
| Casting          | int() float() str() bool(); int("3.5") è ValueError; int(3.9) taglia a 3                   |
| Virgola italiana | .replace(",", ".") e poi float(): il gesto che userete in ogni progetto                    |
| f-string         | :.2f decimali, :>10 :<10 :^10 colonne, :, migliaia, :.1% percentuali                       |
| Costanti         | UPPER_CASE in cima al file; niente magic number; mai il risultato scritto a mano           |
| Scelta del tipo  | Ci faccio i conti? Ha decimali? Sono soldi? CAP, telefono e codici sono testo              |
| Errori           | TypeError tipo, ValueError valore, AttributeError metodo, IndexError fuori; print(type(x)) |

# Grazie.

---

*shaping the skills of tomorrow*

